222 Data Structures and Algorithms

BS(CE)-2k21 LAB

REPORT # 13

Submitted By:

Reg. No.	Name
21-CE-009	Huzaifa Shahbaz

Department of Computer Engineering

HITEC University Taxila

	Presentation (Format(0.5)+ Title (0.5)+ Conclusion(1) +Procedure(2) +Objective(1))	Calculation/ Coding	Observation/ Program Results	Total
Total	5	5	5	15
Obtained				

_____

Lab Instructor.
Kaynat Rana

Experiment #13

Implement Heap using C++

Objective:

The objectives of implementing a heap using C++ are as follows:
1. Create a data structure that allows efficient insertion, deletion, and retrieval of elements.
2. Maintain a partially ordered collection of elements based on specific ordering properties (min-heap or max-heap).
3. Provide quick access to the minimum or maximum element in constant time.
4. Enable fast insertion of new elements while maintaining the heap property.
5. Support efficient deletion of elements while preserving the heap structure.
6. Implement heap sort, an efficient sorting algorithm that leverages the heap data structure.
7. Facilitate the implementation of priority queues for managing elements with varying priorities.
8. Enable event scheduling where elements are sorted based on their scheduled time.
9. Support graph algorithms that require efficient selection of elements based on certain criteria.
10. Provide a versatile data structure that can be used in a wide range of applications requiring efficient element manipulation and ordering.

Equipment Used:

Microsoft Visual Studio

Procedure:

The procedure for implementing a heap using C++ typically involves the following steps:

1. Choose a representation:

Decide on the representation of the heap data structure. One common approach is to use an array to store the elements of the heap, where the indices represent the tree structure.

2. Define the heap class:

Create a class that will encapsulate the heap functionality. Define necessary member variables to store the heap elements, such as the array or vector, and any additional variables to track the size or capacity of the heap.

3. Implement basic operations:

Start by implementing the basic operations required for a heap, such as inserting an element, deleting an element, and accessing the minimum or maximum element. Ensure that these operations maintain the heap property.

4. Build the heap:

If you have an existing collection of elements, you can build the heap by inserting each element into the heap one by one. Alternatively, you can create an empty heap and insert elements as needed.

5. Handle element reordering:

When inserting or deleting elements, you may need to reorder the elements to maintain the heap property. Implement the necessary algorithms to properly adjust the position of elements in the heap.

6. Implement additional operations:

Depending on your specific needs, you may want to add additional operations to your heap

implementation. These can include operations such as checking if the heap is empty, clearing the heap, or updating the value of an element.

7. Test the heap:

Create test cases to validate the correctness of your heap implementation. Test various scenarios, such as inserting elements in different orders, deleting elements, and verifying the ordering properties of the heap.

8. Optimize if needed:

If you encounter performance issues or inefficiencies, consider optimizing your implementation. Look for opportunities to reduce time complexity or improve memory usage, such as using techniques like sift-up or sift-down to efficiently maintain the heap property.

9. Document and maintain:

Finally, document your implementation, including the design choices, assumptions, and any known limitations. Regularly review and maintain the codebase to ensure its reliability and keep up with any future requirements or changes.

Tasks

Task 1:

Implement Heap using C++ by using binary search tree and linked list.

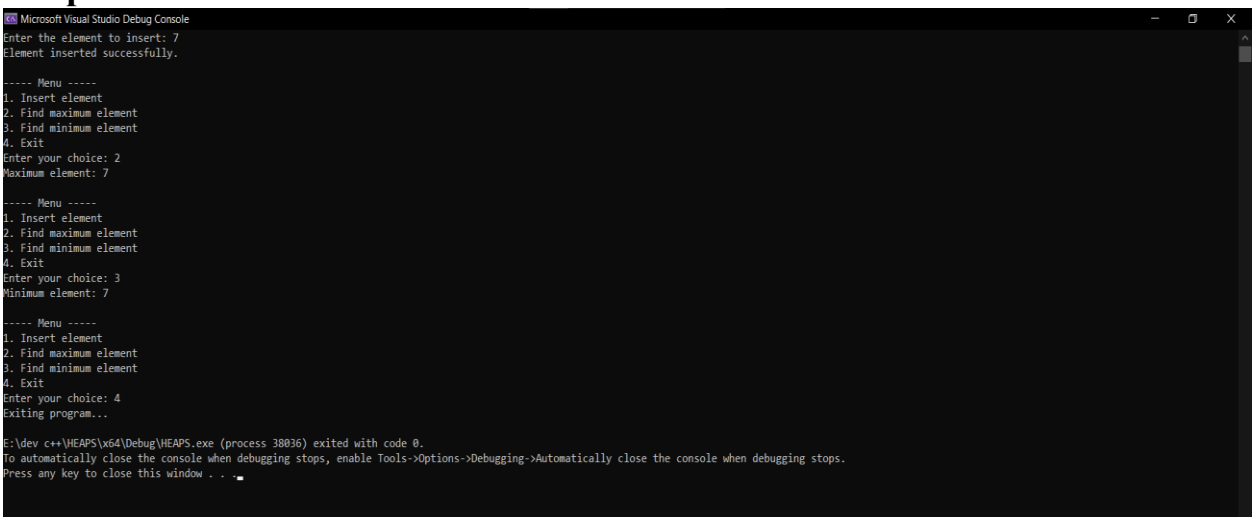
CODE:

```
#include <iostream>
using namespace std;
struct BSTNode
{
    int data;
    BSTNode* left;
    BSTNode* right;
};
struct ListNode
{
    int data;
    ListNode* next;
};
class Heap
{
private:
    BSTNode* root;
    ListNode* head;
public:
    Heap()
    {
        root = NULL;
        head = NULL;
    }
    void insert(int value)
    {
        root = insertBST(root, value);
        head = insertLinkedList(head, value);
    }
    BSTNode* insertBST(BSTNode* root, int value)
    {
        if (root == NULL)
        {
            root = new BSTNode;
```

```
            root->data = value;
            root->left = NULL;
            root->right = NULL;
        }
        else if (value <= root->data)
        {
            root->left = insertBST(root->left, value);
        }
        else
        {
            root->right = insertBST(root->right, value);
        }
        return root;
    }
    ListNode* insertLinkedList(ListNode* head, int value)
    {
        ListNode* newNode = new ListNode;
        newNode->data = value;
        newNode->next = NULL;
        if (head == NULL)
        {
            head = newNode;
        }
        else
        {
            ListNode* current = head;
            while (current->next != NULL)
            {
                current = current->next;
            }
            current->next = newNode;
        }
        return head;
    }
    int findMax()
    {
        BSTNode* current = root;
        while (current->right != NULL)
        {
            current = current->right;
        }
        return current->data;
    }
    int findMin()
    {
        return head->data;
    }
};
int main()
{
    Heap heap;
    int choice, value;
    while (true)
    {
        cout << "----- Menu -----" << endl;
        cout << "1. Insert element" << endl;
        cout << "2. Find maximum element" << endl;
        cout << "3. Find minimum element" << endl;
        cout << "4. Exit" << std::endl;
        cout << "Enter your choice: ";
        cin >> choice;
        switch (choice)
```

```
        {
        case 1:
            cout << "Enter the element to insert: ";
            cin >> value;
            heap.insert(value);
            cout << "Element inserted successfully." << endl;
            break;
        case 2:
            if (heap.findMax() == NULL)
            {
                cout << "Heap is empty." << endl;
            }
            else
            {
                cout << "Maximum element: " << heap.findMax() << endl;
            }
            break;
        case 3:
            if (heap.findMin() == NULL)
            {
                cout << "Heap is empty." << endl;
            }
            else
            {
                cout << "Minimum element: " << heap.findMin() << endl;
            }
            break;
        case 4:
            cout << "Exiting program..." << endl;
            return 0;
        default:
            cout << "Invalid choice. Please try again." << endl;
        }
        cout << endl;
    }
}
```

Output:



Conclusion:

In conclusion, implementing a Heap in C++ enables efficient management and manipulation of elements in a binary tree structure, offering fast access to the maximum or minimum element.